

INF1002 Introduction to Programming

C Report Submission

P6 Group 9

Submission Date: 25 November 2025

Name/ Student Email	Student ID
BONG HONG MING 2403365@sit.singaporetech.edu.sg	2403365
HAFIZHAH MUNIRAH BINTE MAZLAN 2502189@sit.singaporetech.edu.sg	2502189
MUHAMMAD AQIF BIN NOORAZMAN 2501279@sit.singaporetech.edu.sg	2501279
ONG SHI JIE INGMAR 2502020@sit.singaporetech.edu.sg	2502020

Video Demonstration link: <https://youtu.be/h7fi-iJkwww>

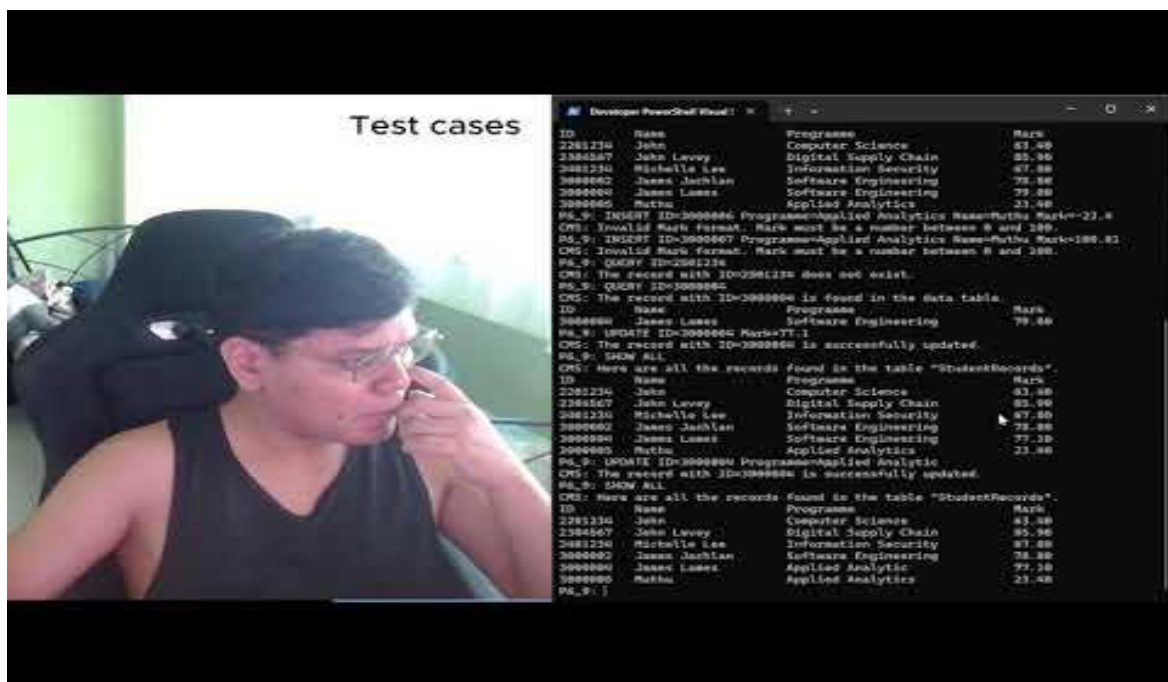


TABLE OF CONTENT

Abstract	3
Introduction	3
System Overview	4
Diagram Data Structure flowchart	5
Data Structure Design	6
Global In-Memory Storage	6
Data Persistence	6
Key Main Core Functions	7
Enhancement features	9
Conclusion of System Design	10
Test Case Results	12

Abstract

The Class Management System (CMS) is a C-language based programme built to manage student academic records. This system is based on a modular file structure to execute database operations and retain changes to the database. This includes the main functions of opening, displaying, querying, updating, deleting and saving records. To add on to that, the CMS encompasses enhancement such as data sorting, summary statistics and displays the calculated grade point average (GPA) of the student's marks. The system ensures thorough validation and saves data using a structured text file format, which for our group is labelled P6_9-CMS.txt

Introduction

The CMS is designed to provide a smooth, command-line interface for administrators or school staff to manage student data effectively. The main goal of the project is to develop a program capable of handling persistent storage of student information while ensuring straightforward user-friendly interaction through specific text commands.

The scope of our system encompasses the full lifecycle of student data supporting core CRUD operations (Creation, Retrieval, Update and Deletion for managing IDs, Names, Programmes, and Marks).

The main feature consists of:

OPEN	Opens and calls the database, reads the records.
SHOW ALL	Displays data in the record in a table manner
INSERT	Inserts new data records into the database. New input will be added at the bottom of the table accordingly. New data consists of student ID, name, programme and mark.
QUERY	Displays if there is student data linked to the given student ID. Data of the student is shown once the student ID is typed as the input. If there is not data linked to the ID, an error message will print.
UPDATE	Updates the data with new record of the existing student ID in the database. New data added includes the name, programme and mark and will be updated accordingly under the student ID.
DELETE	Deletes the record with the selected student ID. The CMS will ask to confirm with the user if the data should be deleted.
SAVE	Saves all edits to the file

Enhancement Features:

SORT	Sorts out student ID based on ascending or descending. Sorts out student Mark based on ascending or descending. Sorts out student Name based on ascending or descending.
SUMMARY	Show summary displays the total number of students, the average mark and the highest and lowest mark linked with the student's name.

CALCULATE GPA (additional unique feature)	Calculates and converts the mark of the student into a grade and their GPA. This feature can show all the student's grade and GPA or a specific student's entry based on their given student ID.
--	--

System Overview

The system is built using a modular architecture, having a main execution loop from the specific functional logic. The program begins in *mainfile.c* by loading existing data from our file, *P6_9-CMS.txt* into memory. The application then goes through an infinite *while(1)* loop, prompting the user with *P6_9:* to input commands.

Our system uses *strcmp* and *strncmp* to parse user's input command. Based on the input, the program routes the execution to the specific function and will call it based on where it is in the source files. (haf.c, ing.c, ingex.c and aq.c). This modular design guarantees that file handling, sorting, querying and validation logics are kept well defined and improved code maintainability.

Execution Flow (User Journey):

From program start → declaration output → command loop → exit/save.

File Flowchart:

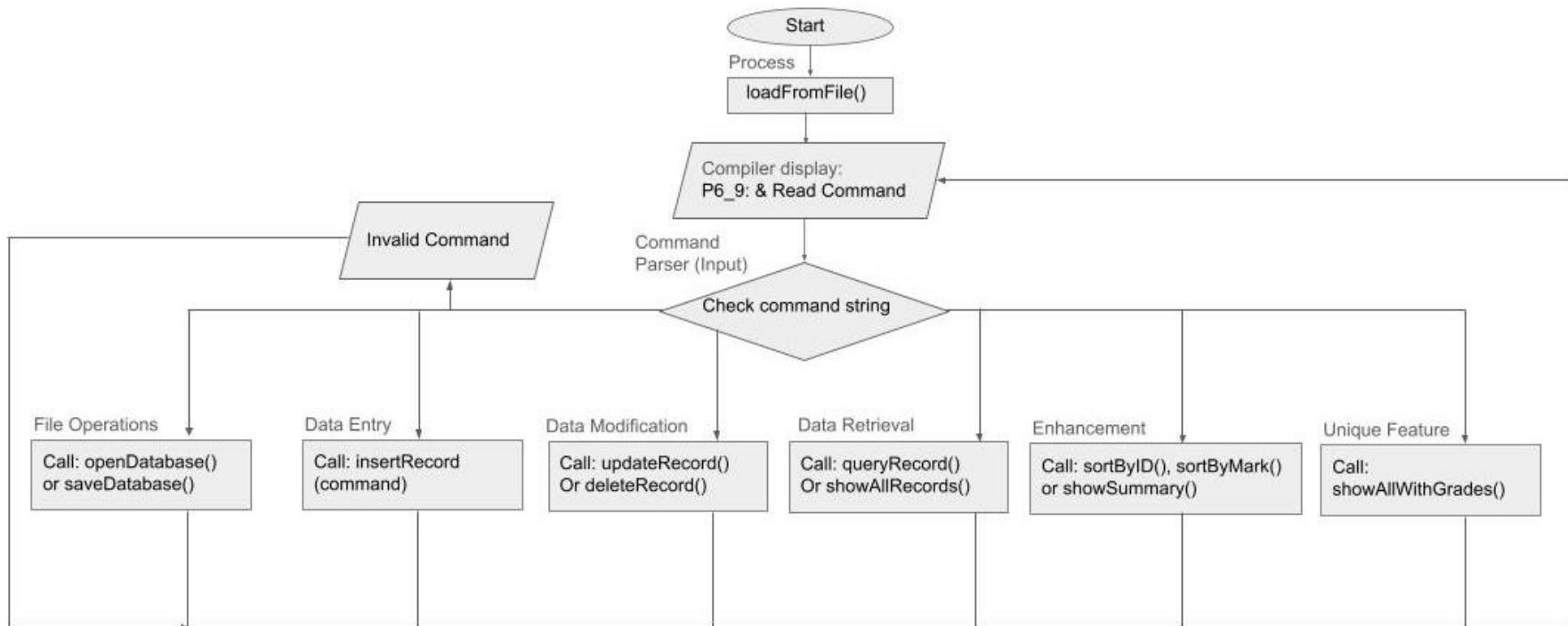
Mainfile.c (Main Loop & Parser) ↓ (*calls*) Feature Modules:

- Ingex.c: Open, Show All, Insert
- Ing.c: Query, summary
- Haf.c: Delete, sorting, gpa calculator
- Aq.c: Update, save, ↓ (*calls*) Helpers / Utils
- Common.c : Find ID
- Aq.c : Validation Functions
- Hfile,h: Data structure

Files	Description
Mainfile.c	The Entry point: Contains the main() loops that asks for user input and command (P6_9:). Command Logic: Parsing logic (checking <i>strcmp</i> ("INSERT") for example). This happens directly inside of mainfile.c and not a separate UI file.
Ingex.c & aq.c	File Operations: ingex.c handles <i>loadFromFile</i> (Open) and aq.c handles <i>saveDatabase</i> (save)
Ingex.c, haf.c, aq.c, ing.c	Core Features: The logic is split up: • Insert (ingex.c) • Delete (haf.c) • Update (aq.c) • Query (ing.c)
Aq.c & common.c	Helpers: aq.c contains validators (isValidID), and common.c contains the helper <i>findRecordIndexByID</i> .

Diagram Data Structure flowchart

The diagram below displays all 10 functions. It shows how the loop is being implemented in the system.



Data Structure Design

The data structure of P6_9 CMS relies on custom-defined structure and a static global array to manage memory efficiently and predictably.

The core of the system is the *StudentRecord* structure defined in *hfile.h*. This data type highlights four distinct fields representing a single student entity:

```
typedef struct {  
    int id;  
    char name [100];  
    char programme [100];  
    float mark;  
} StudentRecord;
```

This structure was chosen to group diverse data types (integer for ID, strings for Name/Programme, and float for Mark) into a single entity. These structures are stored in a global array *extern StudentRecord records [MAX_RECORDS]*; allowing immediate access and manipulation of up to 100 records during run time.

int id: An integer was selected for the Student ID to allow for efficient numerical sorting and arithmetic comparisons (e.g, checking if *id* = 2XXXXXX). This simplifies the logic in functions like *sortByID*.

Char name [100] & char programme [100]: These fields are defined as fixed size character arrays of length 100. This design choice avoids the complexity of dynamic memory allocation (malloc/free) from every string operation. A size of 100 bytes provides safe buffer large enough, to accommodate various names and degree titles without risk of buffer overflow during normal usage.

Float mark: A floating-point number is used to store student marks to strictly support decimal precision (e.g mark=67.67 or 76.67), which is required for our unique feature, the accurate GPA calculation logic.

Global In-Memory Storage

This manages the collection of students; the system uses a Global Array strategy rather than a linked list. This is implemented such as shown:

- *extern StudentRecord records [MAX_RECORDS];*
- *extern int numRecords;*

Static Allocation (records array): The system pre-allocates memory for up to 100 students (MAX_RECORDS) at compile time. This ensures the program has a predictable memory footprint and allows for constant time, $O(1)$, access to any record if the index is known.

Logical Size Tracking (*numRecords*): While the array has a physical capacity of 100, the integer variable *numRecords* acts as a high-water mark, tracking the actual number of active students. Looping throughout the system such as in *showAllRecords* or *findRecordIndexByID*, iterate strictly from 0 to *numRecords*, ensuring the system never accesses uninitialised trash data further down the array.

Data Persistence

The data structure is designed in a way that it maps directly to the row-column format of the P6_9-CMS.txt database file. Our system implements a fixed width parsing strategy where specific byte ranges of a text line are extracted to populate the struct fields:

- ID: Extracted from characters 0-9
- Name: Extracted from characters 10-29
- Programme: Extracted from characters 30-59.

This tight coupling between the in-memory *StudentRecord* struct and the file layout ensures data integrity is maintained during the *loadFromFile* process. When loading, the system, uses string manipulation to parse fixed-width columns, trimming whitespace to ensure clean data entry into the records array.

Key Main Core Functions

LoadFromFile (file ingestion)

Purpose: To initialise the system state by populating the global memory array with persistent data from P6_9-CMS.txt

Logic & Implementation: The loading algorithm utilises a fixed-width parsing strategy rather than simple delimiter splitting. This ensures data integrity even if fields contain spaces (e.g., Programme= Software Engineering)

- The function first consumes and discards the file attributes. (Headers for authors and Table Name) using *fgets*.
- Iterates through the file line-by-line. For each record, it uses *strncpy*, with precise byte offsets to extract data fields: ID (bytes 0-9), Name (bytes 10-29), and Programme (bytes 30-59).
- Essentially, a right trimming algorithm is applied to each extracted string. A while loop iterates backward from the end of the buffer, replacing trailing white space with null terminators (`\0`) to ensure clean strings are stored in memory.

OpenDatabase (System Initialisation)

Purpose: A wrapper function that triggers the loading sequence and confirms system readiness.

Logic & Implementation: This function calls the core *loadFromFile()* logic to populate the memory. Upon completion, it prints a confirmation message to the user:

“CMS: The database file “P6_9-CMS.txt” is successfully opened.”

This separates the backend logic of parsing files from the frontend logic of user notification.

ShowAllRecords (Table Visualisation)

Purpose: To display the entire database in a clean, table format on the console.

Logic & Implementation: This function acts as the primary view for the user.

- The function iterates through the global records array using a for loop bounded by *numRecords*.
- Column Alignment: To ensure the output looks like a professional and neat table. It utilises strict format specifiers in *printf*. For example, `%-20s` ensures the Name field always occupies 20 characters of space, left-aligned, forcing the subsequent “Programme” column to start at the exact same visual position for every row which is in the middle, between the name and mark.

InsertRecord (Command Parsing & Insertion)

Purpose: To parse a complex user command string and append a new student record to the database.

Logic & Implementation: Instead of relying on fixed positions, this function implements a flexible token search using the *strstr* library function.

- The function scans the user's input string to locate substrings "ID=", "Name=", "Programme=", and "Mark=".
- Pointer arithmetic is used to calculate the length of the data between these keys. For example, the length of the Name field is calculated by subtracting the memory address of "Name=" from "Programme=".
- Before insertion, the system pokes *findRecordIndexByID* to enforce a unique constraint, ensuring that there are no duplicate Student IDs are entered.
- The new record is populated at the index *records[numRecords]*, and the counter is incremented. (with source file: *ingex.c*)

QueryRecord (Targeted Retrieval)

Purpose: To locate and display specific student details based on a user given ID.

Logic & Implementation: The function parses the command string to extract the search key/value.

- The function uses *strstr* (*command*, "ID=") to find the pointer address where the ID begins.
- We used *sscanf* with an offset (*idPtr* + 3) to convert the string characters following "ID=" into an integer variable.
- Linear search: It passes this integer to the helper function, *findRecordIndexByID*, which performs a linear O(n) search through the array. If the index is returned as a non-negative, the specific record fields are printed; otherwise, a "not found" error is shown and displayed.

UpdateRecord (Selective Modification)

Purpose: To modify specific attributes (Name, Programme, or Mark) of an existing record without altering the ID.

Logic & Implementation: The update logic is designed to be context aware!

- First validates the existence of the target ID.
- Then scans the input command the user types in to determine which field the user intends to update. The code uses an if-else if, ladder checking for the presence of the student's "Mark=", "Programme=", or "Name=".
- Strict Input Validation is applied immediately before the update. Such as if the user attempts to update a mark, *isValidMark* is called to ensure the value is a floating-point number between 0 to 200. This prevents data corruption from invalid user input (as referenced from *aq.c* file.)

DeleteRecord (Array Compaction)

Purpose: To permanently remove a student record and maintain memory continuity.

Logic & Implementation: Deleting from a static array requires managing the “gap” left by the removed element.

- The function locates the target index and displays the record details, prompting the user for a “Y/N” confirmation vis *scanf*.
- Upon confirmation, the system executes a left-shift algorithm: a loop runs from the deleted index *i* up to *numRecords - 1*, executing *records[i] = records[i + 1]*.
- This overwrites the deleted record with the next one and shifts all subsequent records down by one, accordingly and effectively closing the gap and keeping the data contiguous in memory (as referenced from haf.c)

SaveDatabase (Data Persistence)

Purpose: To serialise the current state of the in-memory array and write it permanently to P6_9-CMS.txt.

Logic & Implementation: The function opens the file in write mode (“w”), which intentionally truncates existing content to ensure the file reflects the exact current state of the program.

- Metadata writing: First writes the required system headers: “Database Name”, “Authors”, and “Table Name”) using *fprintf*. This maintains the specific file structure required for the *openDatabase* function to parse correctly later.
- Formatted serialisation: The functions iterate through the records array from 0 to *numRecords*. It uses *fprintf* with precise width specifiers (e.g., %-10d, %-20s) to ensure the text file remains human-readable and column-aligned.
- Error handling: Validates the file pointer (if (! file) to prevent a crash if the system lacks write permission.

Enhancement features

Advanced Sorting (qsort Implementation)

The system implements a highly efficient sorting mechanism using C standard library function – *qsort*. This approach is superior to basic bubble sorts, quadratic ($O(n^2)$), as it typically utilises *QuickSort* ($O(n \log n)$) linear logarithmic.

- Design: Four distinct comparator functions were written (*compareByIDAsc*, *compareByIDDesc*, *compareByMarkAsc*, *compareByMarkDesc*). These functions cast void* pointers to *StudentRecord** and return an integer difference, allowing the system to sort the entire database dynamically based on user preference (ref: haf.c).
- Sturdy Input Validation: To prevent “garbage-in, garbage-out” scenarios, a dedicated validation module was implemented in aq.c.
- *IsValidID*: Iterates through the string to ensure every character is a digit (0-9).
- *IsValidProgramme* & *IsValidName*: Uses *isalpha* to permits only alphabetic characters, spaces, and hyphens, rejecting special symbols or number in names.

- *IsValidMark*: Uses *strtof* to parse the float and strictly enforces a range check ($0.0 \leq \text{mark} \leq 100.0$) (ref: aq.c)

showSummary (Statistical Analysis)

This system computes and displays descriptive statistics for the class: Total Students, Average, Highest, and Lowest Mark). The algorithm performs a single-pass $O(n)$ linear analysis to calculate all statistics simultaneously.

- Initialisation: Sets the initial *highestMark* and *lowestMark* to the values of the first *student* (*records [0]*) to establish a baseline.
- Accumulation: Inside the loop, it adds every mark to a running *totalMarks* variable.
- Conditional updates: For every record, it compares the mark against the current high/low. If a student's mark is higher than *highestMark*, the variables for the mark and the student's name are updated immediately.
- Final Calculation: After the loop, it computes the mean ($\text{totalMarks} / \text{totalStudents}$) and prints the summary block.

GPA calculator (Unique Feature)

This feature in the CMS allows the ability to perform academic analytics by converting raw marks into standardised Grade Point Averages (GPA).

- Custom Grading Logic: The system implements a custom translation layer in *getGPAFromMark* and *getGradeFromMark*. It uses a tiered conditional structure to map specific mark ranges to a 4.0 scale: mark $\Rightarrow$ 85.0 maps to 4.0/A, while marks between 60.0 and 63.0 map to 1.7/C-). This logic allows for precise academic evaluation.
- Class-Wide Analytics (*showAllWithGrades*): This function aggregates data to provide a overview of class performance.
- It iterates through the entire dataset, calculating the GPA for every student immediately.
- Accumulates a running total ($\text{totalGPA} += \text{gpa}$) to compute and display the Class Average GPA at the end of the report. This transforms the CMS from a simple data repository into an analytical tool for educators.

Conclusion of System Design

Our design of the P6_9 Class management system prioritises structural modularity, memory predictability and data integrity. By compartmentalising functional logic into clear, and distinct source files (e.g, haf.c for sorting/ deletion, aq.c for validation/ updates, ing.c for summary/ query), the system achieves a clean separation of concerns that simplifies debugging and future scalability and maintenance.

The architectural decision to utilise a static global array (*records [100]*) ensures the application maintains a low and stable memory footprint, which eliminates risks associated with dynamic allocation in a restricted environment. Furthermore, we found that the implementation of tough input validation and fixed-width file parsing guarantees that the system remains resilient against user error and data corruption.

Ultimately, the integration of algorithmic enhancement such as *qsort* for ordering and custom logic for enhancement features, transforms the application from a simple storage utility into a comprehensive tool for academic management.

Test Case Results

A total of 10 test cases were created to check all the main features of the program, such as **INSERT, UPDATE, DELETE, QUERY, SHOW, SORT, and file handling**. These test cases also included important edge cases like duplicate IDs, invalid inputs, and boundary values. During the testing phase, a few bugs were discovered and analysed. After making the necessary fixes, the program now runs correctly and behaves as expected.

Test CaseID	Description	Input Command(s)	Expected Output	Reason for Test	Actual Result
OPN-01	Missing File	OPEN	CMS: Error opening file	Ensure file is present	PASS
INS-DUP-001	Duplicate ID Detection	INSERT ID=100001 Name=Ding Yi Programme= Applied AI Mark=89.1 INSERT ID=100001 Name=Guo Qiang	First: CMS: A new record with ID=100001 inserted. Second: CMS: Record with ID=100001 already exists.	Ensure IDs are not duplicated to maintain data accuracy	PASS
INS-VAL-001-01	Missing/Partial Input	INSERT Name=Thomas Programme=Information Security Mark=79	CMS: Missing ID field.	Check that all required fields are provided	PASS
INS-VAL-001-02		INSERT ID=100001 Programme=Information Security Mark=79	CMS: Missing Name field.		PASS
INS-VAL-001-03		INSERT ID=100001 Name=Thomas Mark=79	CMS: Missing Programme field		PASS
INS-VAL-001-04		INSERT ID=100001 Name=Thomas Programme=Information Security	CMS: Missing Mark Field.		PASS
UPD-VAL-001	Mark Range Validation (>100)	UPDATE ID=100001 Mark=105.00	CMS: Invalid Mark format. Mark must be between 0 and 100.	Check academic mark are within range (0-100 only)	PASS
UPD-VAL-002	Mark Range Validation (<0)	UPDATE ID=100001 Mark=-1.0			PASS

UPD-VAL-003	Name Format Validation (Numbers)	UPDATE ID=100001 Name=John123	CMS: Invalid Name format. Name must contain only letters, spaces, hyphens.	Check that name validation rejects numbers	PASS
UPD-NAME-001	Update Name Field Only	UPDATE ID=100001 Name=Tisha	CMS: The record with ID=2000001 is successfully updated.	Verify Name field updates independently without affecting other fields	PASS
UPD-VAL-004	Update All Fields	UPDATE ID=10001 Name=John Programme=AI Mark=79 UPDATE ID=10001 Mark=79 Name=John Programme=AI UPDATE ID=10001 Mark=79 Name=John	CMS: The record with ID=10001 is successfully updated.	Verify Name field updates independently without affecting other fields	PASS
DEL-ERR-001	Delete Non-Existent Record	DELETE ID=99999	CMS: The record with ID=99999 does not exist.	Check error handling for missing records	PASS
DEL-ERR-02	Delete invalid input	DELETE ID=X999	CMS: Invalid input. Please provide an ID number.	Check for invalid input	PASS
GPA-CALC-001	GPA Calculation - All Grade Boundaries	INSERT ID=200001 Name=Jason Quek Programme=Software Engineering Mark=85.9 INSERT ID=200002 Name=Lydia Masjid Programme=Computer Engineering Mark=63.0 INSERT ID=200004 Name=Samantha Williams Programmer=Applied Analytics Mark=49.0 SHOW ALL WITH GRADES	ID=200001: Grade A, GPA=4.0 ID=200002: Grade C, GPA=2.0 ID=200004: Grade F, GPA=0.0	Verify GPA accuracy at critical grade boundaries	PASS

FILE-PERSIST-001	Data Persistence (Save & Load)	INSERT ID=300001 Name=Ahmad Programme=Applied AI Mark=75.5 SAVE OPEN QUERY ID=300001	After reopening: Record found with all data intact (ID, Name, Mark)	Verify file I/O saves and loads data correctly	PASS
FILE-SAVE-001	Save without textfile	SAVE	CMS: No records to save	Verify if the file saves without any files	PASS
SORT-FUNC-001	Sort by Mark (Descending)	INSERT ID=2000001 Name=Nisha Programme=Applied AI Mark=70 INSERT ID=2000002 Name=Jun Hao Programme=Applied AI Mark=76 INSERT ID=2000003 Name=Ding Yi Programme=Applied AI Mark=84.1 SHOW ALL SORT BY MARK DESCENDING	Records displayed in order: 84.1, 76, 70 (descending)	Verify SORT functionality displays correctly ordered results	PASS
DISPLAY-FUNC-001	Show All & Summary Statistics	INSERT ID=2000001 Name=Nisha Programme=Applied AI Mark=70 INSERT ID=2000002 Name=Jun Hao Programme=Applied AI Mark=60 INSERT ID=2000003 Name=Ding Yi Programme=Applied AI Mark=92.4 SHOW SUMMARY	CMS: Summary Total Students: 3 Average Mark: 74.13 Highest Mark: 92.40 (Student: Ding Yi) Lowest Mark: 60.00 (Student: Jun Hao)	Verify display functions and statistics calculation	PASS

EDGE-BOUND-001	Boundary Mark Values (0, 50, 100)	INSERT ID=2000001 Name=Nisha Programme=AAI Mark=0 INSERT ID=2000002 Name=Jun Hao Programme=AAI Mark=50 INSERT ID=2000003 Name=Ding Yi Programme=AAI Mark=100 SHOW ALL INSERT 3 records with Mark=0.00, Mark=50.00, Mark=100.00 SHOW ALL	All records inserted, marks displayed exactly as entered ID, Name, Programme, Mark 2000001 Nisha AAI 0.00 2000002 Jun Hao, AAI 50.00 2000003 Ding Yi AAI 100.00	Verify boundary values handled correctly without loss of precision	PASS
----------------	-----------------------------------	--	---	--	------

Test Insights and Bug Discoveries:

1. Mark Boundary Testing

- Initially, marks less than 0 or greater than 100 were being accepted into the system. We created *isValidMark()* to properly reject values outside the range [0.0, 100.0].

2. Case Sensitivity Problem

- The UPDATE function was case-sensitive for field names. Commands like "mark=" weren't recognized as "Mark=". We fixed this by adding fallback checks for lowercase variants in the field detection logic.

3. Field Order Handling Bug

- insertRecord()* originally assumed fields would always appear in a fixed order (Name before Programme). When we tested with "Programme Before Name", the parser failed to extract values correctly. We fixed this by implementing dynamic boundary detection that calculates field positions based on actual token locations in the input string.

4. Empty Field Validation

- We found that commands like "INSERT ID=1234570 Name= Programme=IT Mark=85" were being accepted despite having an empty Name field. We added explicit checks to detect empty values immediately after the = sign and reject them.

Team Member's Workload Distribution

HAFIZHAH MUNIRAH BINTE MAZLAN	• Functions: Delete, Sorted, GPA calculator • Integration of files • Video editing • Report
MUHAMMAD AQIF BIN NOORAZMAN	• Functions: Update, Save • Test case design and testing of test cases. • Report refinement • Overall code refinement
ONG SHI JIE INGMAR	• Functions: Open, show all, Insert, Query, Show Summary. • Overall code refinement • Test case refinement
GENERATIVE AI	• Debugging of code • Project guidance

Declaration:

In accordance with SIT's policy, which strictly prohibits copying source code or assessment solutions from other persons, AI or external places, it is the student's responsibility to guarantee that their work is original and inaccessible to others; where plagiarism is detected, all involved assessments will receive a zero mark. We hereby declare that we have not copied code from other, AI-generated sources or external places, nor have we shared our code or uploaded it for public access, and we agree not to do so in the future. We accept that our project will receive a zero mark if plagiarism is detected and agree not to disclose any group project materials to others. This declaration is made by group P6_9: compromising team members Hafizhah Munirah Binte Mazlan, Muhammad Aqif Bin Noorazman, and Ong Shi Jie Ingmar, on 25/11/2025.

Total valid number of pages used: 14.

Appendix

Reference List:

References:

<https://www.livecodeprogramming.com/tutorials/c/projects/student-record-management-c.php>

<https://www.geeksforgeeks.org/c/basics-file-handling-c/>

<https://www.geeksforgeeks.org/c/how-to-create-dynamic-array-of-structs-in-c/>